

Can LLMs Maintain Fundamental Abilities under KV Cache Compression?

Xiang LIU[♡] Zhenheng TANG[♣] Hong CHEN[♡] Peijie DONG[♡] Zeyu LI[♡]
 Xiuze ZHOU[♡] Bo LI[♣] Xuming HU^{♡†} Xiaowen CHU^{♡†}
[♡] The Hong Kong University of Science and Technology(Guangzhou)
[♣] The Hong Kong University of Science and Technology
 {xliu886, hchen763, pdong212, zli755, xzhou154}@connect.hkust-gz.edu.cn
 {zhtang.ml, bli}@cse.ust.hk {xuminghu, xwchu}@hkust-gz.edu.cn

Abstract

This paper investigates an underexplored challenge in large language models (LLMs): the impact of KV cache compression methods on LLMs’ fundamental capabilities. Although existing methods achieve impressive compression ratios on long-context benchmarks, their effects on core model capabilities remain understudied. We present a comprehensive benchmark KVFundaBench to systematically evaluate the effects of KV cache compression across diverse fundamental LLM capabilities, spanning world knowledge, commonsense reasoning, arithmetic reasoning, code generation, safety, and long-context understanding and generation. Our analysis reveals several key findings: (1) *Task-Dependent Degradation*; (2) *Model-Type Robustness* (3) *Prompt Length Vulnerability*; (4) *Chunk-Level Superiority*; (5) *Prompt-Gain Sensitivity*; (6) *Long-Context Generation Sensitivity*. Based on our analysis of attention patterns and cross-task compression performance, we propose ShotKV, a novel compression approach that distinctly handles prefill and decoding phases while maintaining shot-level semantic coherence. Empirical results show that ShotKV achieves 9%-18% performance improvements on long-context generation tasks under aggressive compression ratios.

1 Introduction

The evolution of Large Language Models (LLMs) to process large documents for tasks such as answering and summarizing questions [1–6], spurred by breakthroughs in system architectures [7–10] and model design [11–14], has significantly increased GPU memory demands during inference [15–20], making the development of efficient key value (KV) cache compression strategies a critical focus for LLM deployment and optimization.

To address this, numerous studies have proposed selective token retention strategies [10, 21–29], with pioneering works such as H2O [21] and SnapKV [22] showing that retaining approximately 50% of KV cache entries can balance model performance with significant memory savings. However, current research primarily evaluates these methods in long-context scenarios such as LongBench [30, 31] and Need-In-A-Haystack (NIAH) [32], leaving their impact on a spectrum of fundamental LLM capabilities—such as *arithmetic reasoning*, *world knowledge*, *commonsense reasoning*, and *safety*, particularly concerning their distinct attention patterns under compression, largely unexplored. To this end, we introduce **KVFundaBench**, a benchmark designed to systematically assess the effects of KV cache compression across these diverse fundamental capabilities and their underlying attention dynamics. The benchmark includes 5 categories of tasks: *world knowledge*, *commonsense*

[†]Corresponding Author.

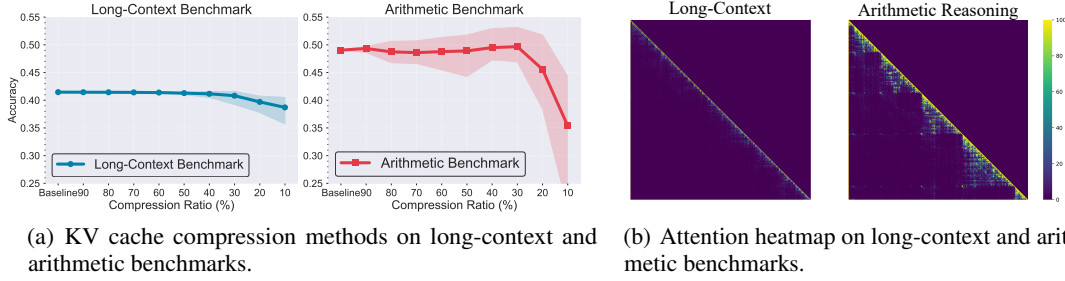


Figure 1: KV cache compression methods on long-context and arithmetic benchmarks. (a) Arithmetic benchmark shows more performance degradation than long-context benchmark. (b) Long-Context benchmark shows more sparsity in attention heatmap.

reasoning, arithmetic reasoning, code generation, and safety. Our comprehensive evaluations using KVFundabench reveal several critical findings: we observe, as shown in Figure 1, that arithmetic reasoning tasks suffer significantly higher performance degradation under compression compared to long-context tasks, and that attention patterns in long-context scenarios exhibit notably higher sparsity. These initial results suggest that existing evaluation frameworks, which focus predominantly on long-context performance, may not adequately capture the full impact spectrum. Our KVFundabench reveals several key findings: (1) *Task-Dependent Degradation*: Performance degradation is highly task-dependent, with arithmetic reasoning tasks showing particular sensitivity to aggressive compression; (2) *Model-Type Robustness*: Multi-step reasoning LLMs demonstrate higher compression robustness compared to instruction-tuned models; (3) *Prompt Length Vulnerability*: Shorter prompts are more vulnerable to compression effects; (4) *Chunk-Level Superiority*: Chunk-level compression strategies show superior performance on complex long-context reasoning tasks; (5) *Prompt-Gain Sensitivity*: Tasks with larger prompt-based performance gains exhibit higher compression sensitivity; and (6) *Long-Context Generation Sensitivity*: Long-context generation tasks are particularly sensitive to compression. These findings provide valuable insights into the relationship between compression methods and model capabilities, motivating our development of **ShotKV**, which is a new KV cache compression method with separate compression methods for prefill and decoding phases.

We hope our work can provide the research community with insightful perspectives on the impact of KV cache compression on LLMs. Our main contributions are summarized as follows:

- Introduce **KVFundaBench** to systematically evaluate the effects of KV cache compression across diverse fundamental LLM capabilities, we demonstrate that task-specific sensitivity to compression varies significantly, with performance degradation ranging from 1% to 40%.
- Our systematic investigation reveals multiple critical factors influencing compression sensitivity, including model training dynamics, prompt length characteristics, task-specific requirements, long-context reasoning, and long-context generation capabilities.
- We introduce **ShotKV**, an innovative compression framework that distinctively manages the prefill and decoding phases while maintaining the semantic integrity of the shot level.

2 Preliminary

In this section, we provide comprehensive preliminaries of KV cache compression and LLM evaluation.

Key-Value Cache in LLMs With the increasing long-context capabilities of LLMs, the Key-Value (KV) cache has become crucial for improving inference efficiency. During LLM inference, the KV cache stores intermediate computation results to avoid redundant calculations. For a given input sequence $x = (x_1, x_2, \dots, x_n)$, each transformer layer l maintains its key cache $K^l = (k_1^l, k_2^l, \dots, k_n^l)$ and value cache $V^l = (v_1^l, v_2^l, \dots, v_n^l)$, where $k_i^l, v_i^l \in \mathbb{R}^d$ represent the key and value vectors for token x_i at layer l .

KV Cache Compression KV cache compression aims to reduce memory usage by selectively storing or merging cached vectors. A compression operation can be denoted as $C(K, V) = (K', V')$,

where K' and V' are compressed caches with size $m < n$, where C is the compression method, m is the number of retained tokens, and n is the original number of tokens. The core idea is token selection - identifying and retaining important tokens based on attention patterns or other metrics while discarding less important ones. The compression ratio $r = m/n$ indicates how aggressively the cache is compressed, where a smaller ratio means more aggressive compression.

Evaluation Protocol To thoroughly evaluate the impact of KV cache compression on LLMs' capabilities, we assess five benchmark categories: world knowledge, commonsense reasoning, arithmetic reasoning, code generation, and safety.

For each task category and compression method C , we calculate the relative performance change as follows:

$$\Delta P = \frac{P_C - P_{\text{base}}}{P_{\text{base}}} \quad (1)$$

where P_C and P_{base} represent the performance scores with and without compression, respectively.

3 Benchmark Design

3.1 Benchmark Setups

In this section, we will introduce the KVFundaBench setups, including the datasets, models, and evaluation environment.

Datasets To evaluate the performance of KV cache compression on LLMs' overarching capabilities, we assess five benchmark categories: **World Knowledge (WK)** using MMLU [33], measured by accuracy; **CommonSense Reasoning (CSR)** using CommonsenseQA [35], evaluated through multiple-choice accuracy; **Arithmetic Reasoning (AR)** using GSM8K [34], assessed by solve rate; **Code Generation (CG)** using HumanEval [36], measured by pass@1 rate on test cases; and **Safety (SA)** using JailBreakV [37], evaluated by attack success rate. Furthermore, we test the performance of KV cache compression on LongGenBench [38], a **long-context generation (LG)** benchmark. Table 1 summarizes the statistics of the data set.

Table 1: A summarized view of KVFundaBench

DATASET	TASK TYPE	# TEST
MMLU [33]	World Knowledge	14,079
GSM8K [34]	Arithmetic	1,319
CSQA [35]	Commonsense	1,221
HumanEval [36]	Code Generation	164
JailBreakV [37]	Safety	28,000
LongGenBench [38]	Long-Context Generation	23,000

Models We conduct experiments on a series of LLMs, including LLaMA-3.1-8B, LLaMA-3.1-8B-Instruct [39], Mistral-7B-Instruct [40], and multi-step reasoning LLM DeepSeek-R1-Distill-Llama-8B [41].

KV Cache Compression Methods To thoroughly investigate the potential impact on KV cache compression methods, we select the following methods: StreamingLLM [10], SnapKV [22], H2O [21], PyramidKV [24], PyramidInfer [26], and ChunkKV [42].

Hyperparameters For the experiments on observation 1, 2 we set the normal shot number as in the original paper. For the experiments on observation 3, 4, 5, the shot number are dependent on the experiment settings. More details are shown in Appendix B.1.

3.2 Attention

Pattern Analysis on KVFundaBench

To better understand the task-specific sensitivity to KV cache compression observed in our experiments, we analyze the attention patterns of LLMs on KVFundaBench. This analysis provides

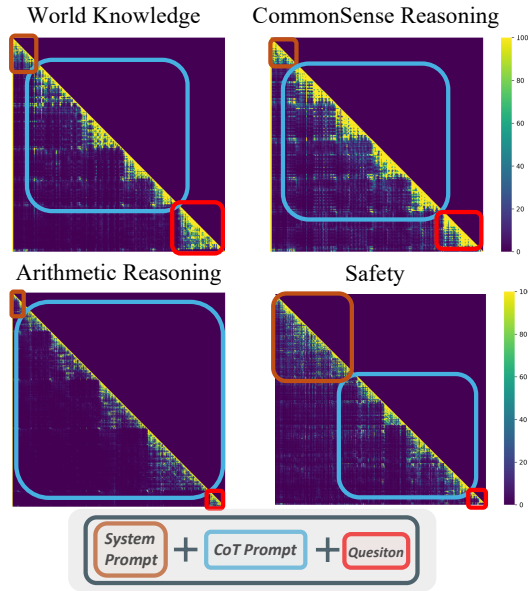


Figure 2: Attention heatmap on different tasks.

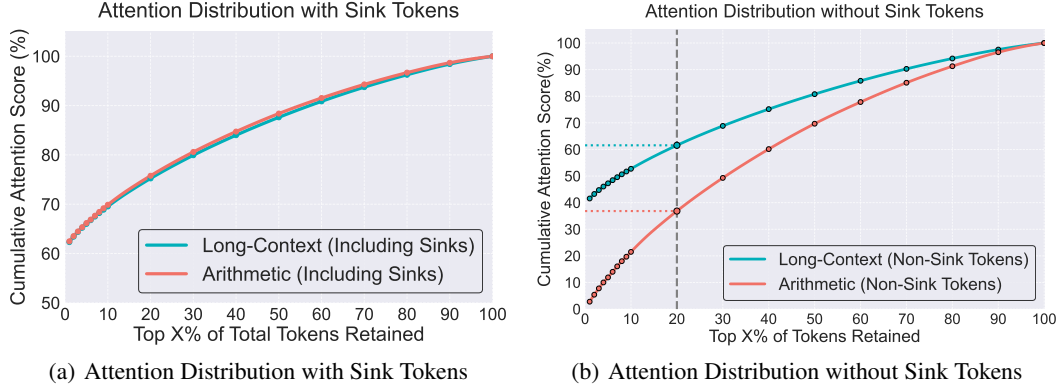


Figure 3: Cumulative attention score distribution for Long-Context and Arithmetic. (a) Overall distribution including initial sink tokens, showing high initial concentration. (b) Distribution without sink tokens (first 4 tokens removed), revealing that Arithmetic’s non-sink attention is more diffuse compared to Long-Context’s.

key insights into why different tasks respond differently to compression, from two perspectives: (1) task-specific attention patterns; and (2) attention distributions.

As shown in Figure 2, our benchmark reveals different attention behaviors in tasks: world knowledge and common sense reasoning tasks exhibit **universal attention distributions**, while arithmetic reasoning and safety tasks display more **specialized patterns**. In particular, arithmetic reasoning tasks show increased attention sparsity (i.e. focused attention on individual prompt examples), and safety tasks concentrate attention on system prompts, in contrast to the more uniform attention distribution seen in world knowledge and commonsense reasoning.

To further investigate the attention dynamics that might explain the task-specific sensitivities to KV cache compression, we analyzed the cumulative attention score distributions, as illustrated in Figure 3. Figure 3(a) depicts the overall attention distribution, which includes the initial sink tokens [10]. In this view, both long-context and arithmetic tasks demonstrate a very similar pattern: a steep initial rise where the top 1% of tokens capture over 60% of the total attention mass. This highlights the predominant role of sink tokens in attracting attention, regardless of the specific task.

However, a more distinct pattern emerges when these initial sink tokens (specifically, the first four tokens) are excluded from the analysis, as shown in Figure 3(b). Within the remaining non-sink tokens, the attention distribution for arithmetic tasks becomes notably more diffuse, with a slower accumulation of attention mass. For instance, the top 20% of non-sink tokens in arithmetic cover only about 37% of the attention within their own non-sink group. In contrast, long-context’s non-sink tokens exhibit a relatively more concentrated attention profile, where the top 20% of its non-sink tokens capture approximately 61.5% of the attention within their non-sink set. This divergence suggests that while sink tokens provide a common, strong attentional anchor, the

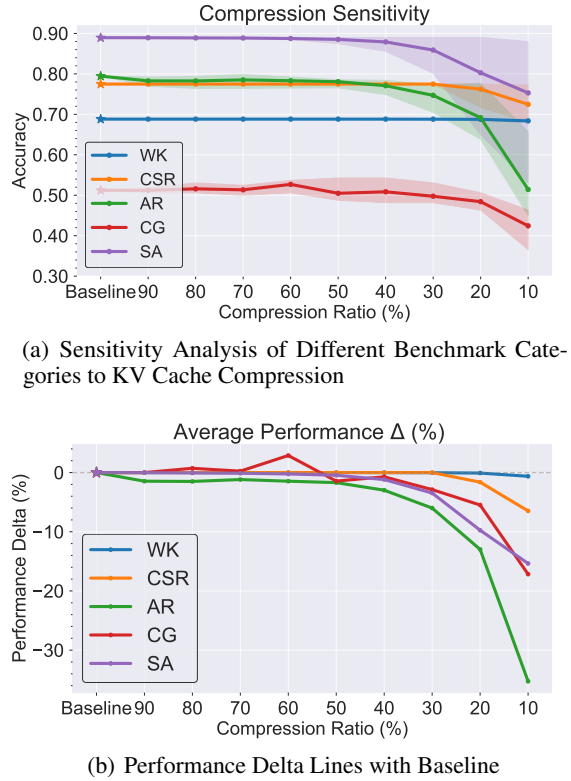


Figure 4: Sensitivity Analysis of Different Benchmark Categories to KV Cache Compression. The performance delta lines are calculated by Equation (1).

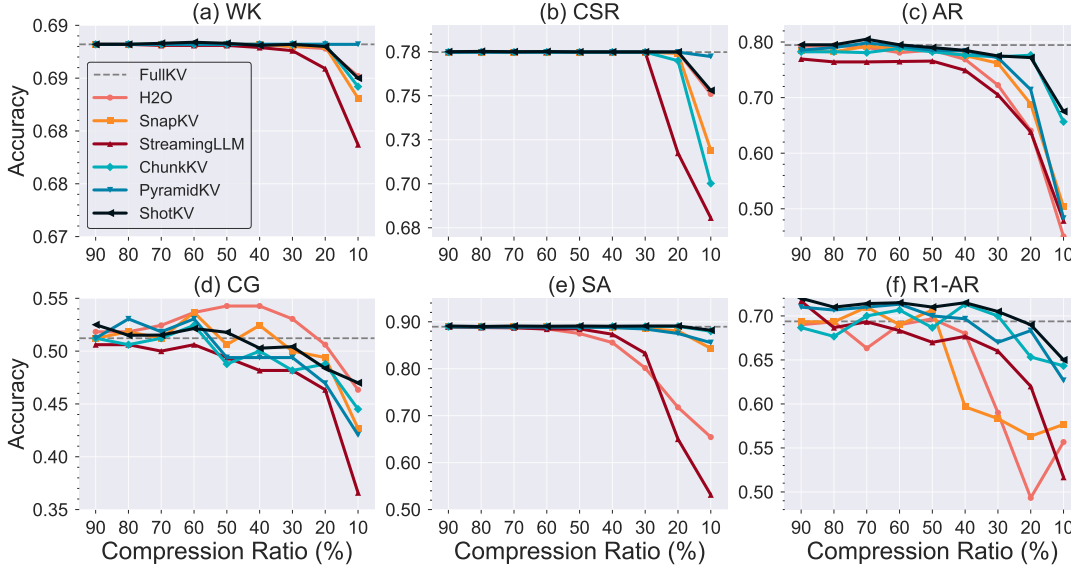


Figure 5: Performance Comparison of KV Cache Compression Methods on KVFundaBench. Results for R1-AR (f) were obtained using the DeepSeek-R1-Distill-Llama-8B model. ShotKV is our proposed method; details can be found in Section 4.

subsequent distribution of attention across task-relevant (non-sink) tokens varies. The more diffuse attention in arithmetic’s non-sink tokens implies a reliance on a broader set of contextual cues for its structured reasoning, potentially making it more vulnerable when compression begins to impact these non-sink tokens.

These detailed analyses of attention distributions (Figure 2 and Figure 3) reveal that LLMs engage different contextual information and attention strategies when performing long-context tasks versus tasks requiring fundamental abilities such as arithmetic reasoning. This highlights the necessity of evaluating KV cache compression beyond long-range dependencies to specifically assess its impact on diverse fundamental capabilities, owing to their distinct attentional mechanisms.

3.3 Results and Analysis

In this section, we present the results and an analysis of the experiments. For detailed results, see Appendix B.2.

Evaluation Environment We use the lm-evaluation-harness [43] library to load the models and evaluate the performance. The evaluation environment is a NVIDIA A40 GPU server.

Observation 1. Task-Dependent Degradation: KV cache compression methods show task-dependent performance degradation, WK and CSR are more robust to KV cache compression. As demonstrated in Figure 4, all tasks maintain stable performance at compression ratios above 40%, but exhibit distinct degradation patterns below this threshold. *Arithmetic reasoning*, *code generation*, and *safety* tasks demonstrate the highest compression sensitivity, characterized by precipitous performance declines. Figure 5 illustrates the detailed performance impact of various KV cache compression methods across different tasks. This degradation is most pronounced in *arithmetic reasoning* (c), where performance deteriorates significantly below the compression ratio of 20%, with precision dropping from approximately 0.75 to below 0.5. Among the evaluated methods, ChunkKV [42] and PyramidKV [24] consistently demonstrate superior stability in most tasks, while StreamingLLM [10] exhibits increased sensitivity to aggressive compression. Additionally, *R1-Arithmetic reasoning* (f) indicates that reasoning LLMs demonstrate enhanced robustness to KV cache compression. Highlighting *World Knowledge* and *Common Sense Reasoning* as the most robust tasks, indicating that these tasks are less sensitive to KV cache compression.

Observation 2. Model-Type Robustness: Multi-step reasoning LLMs are more robust to KV cache compression. Figure 7 presents a comparative analysis of LLaMA-3.1-8B across its base (w/o instruct tuned), instruct-tuned, and DeepSeek-R1 distilled variants, illustrating their averaged performance in five compression methods with confidence intervals. Although all three variants exhibit performance degradation at low compression ratios, their degradation trajectories differ significantly. The R1 distilled model demonstrates superior stability, maintaining performance around 0.60 even at a compression ratio 10%. Although the instruct-tuned model achieves a higher initial accuracy (0.8),

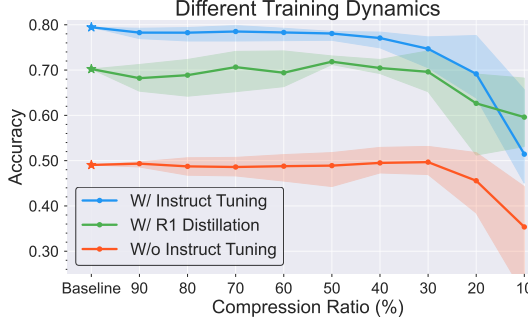


Figure 7: Performance Comparison of KV Cache Compression Methods on different training dynamics on *Arithmetic Reasoning*

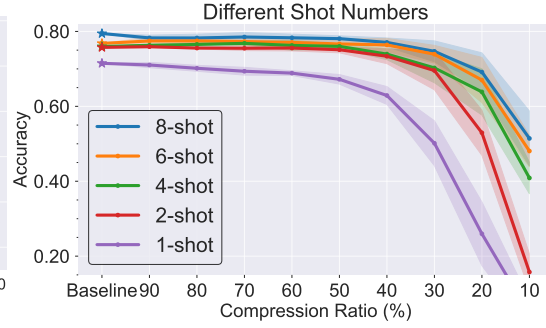


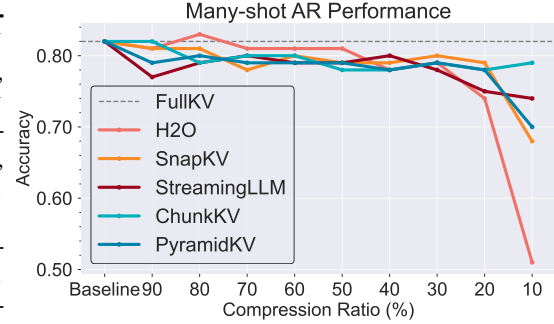
Figure 8: Average Performance Across Different Shot Numbers

it exhibits heightened compression sensitivity, with performance deterioration beginning at 30% compression ratio and declining sharply to approximately 0.5 at 10% ratio. These findings suggest that while multi-step reasoning LLMs demonstrate enhanced robustness to KV cache compression, and instruct-tuning improves overall model performance, the latter may inadvertently increase model vulnerability to aggressive compression, particularly at compression ratios below 30%.

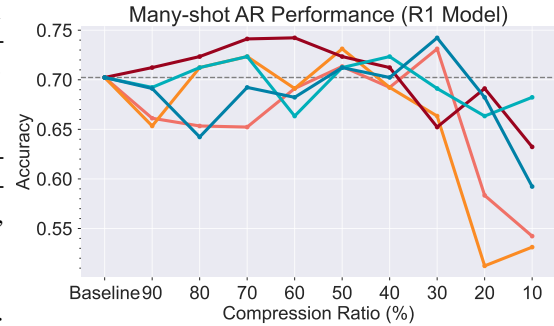
Observation 3. Prompt Length Vulnerability: Shorter prompts are more vulnerable to KV cache compression. As illustrated in Figure 8, the effect of KV cache compression is markedly different with varying prompt lengths (shot numbers). Scenarios with fewer shots (for example, one-shot and two-shot) demonstrate heightened sensitivity to compression; their performance degrades more precipitously below a compression ratio of 30% compared to scenarios with a greater number of shots (e.g., 4-8 shots). For example, in 1-shot settings, performance decreases from 0.5 to 0.05 as the compression ratio decreases from 30% to 10%. In contrast, 8-shot settings experience a less severe reduction, from 0.75 to 0.5, under the same compression conditions. This suggests that prompts with more shots, by virtue of containing more contextual examples, offer a richer set of reference points for the model. Consequently, the model’s reliance on any single example being perfectly preserved in the compressed KV cache is reduced, leading to greater robustness against aggressive compression.

Observation 4. Chunk-Level Superiority: Chunk-level compression is more effective for long-context structured reasoning tasks. Inspired by Agarwal et al. [44], we consider many-shot in-context learning as a long-context reasoning task, which is more complex than existing long-context benchmarks, such as LongBench

and NIAH. Figure 6 shows the performance of KV cache compression methods on a 50-shot GSM8K task, where the prompt length exceeds 4K tokens. From the figure, we observe that ChunkKV [42] demonstrates the most stability when the compression ratio is below 10% on both LLaMA-3.1-8B-Instruct and DeepSeek-R1-Distill-Llama-8B, indicating that in more complex long-context arithmetic reasoning tasks, chunk-level retention is more effective at preserving semantic information. Highlighting the effectiveness of chunk-level compression for long-context structured reasoning tasks.



(a) Many-shot *Arithmetic Reasoning* on LLaMA3.1-8B-Instruct



(b) Many-shot *Arithmetic Reasoning* on DeepSeek-R1-Distill-Llama-8B

Figure 6: Many-shot scenario on KV cache compression

Observation 5. Prompt-Gain Sensitivity: KV cache compression significantly reduces performance gains from ICL and CoT. As shown in Table 2, different tasks exhibit varying levels of performance improvement from zero-shot to CoT prompting. *Arithmetic reasoning* shows a dramatic improvement of 50.41%, while *World Knowledge* demonstrates a more modest gain of 6.20%. From Figure 4, we find that tasks with larger CoT improvements, such as *Arithmetic reasoning*, are more sensitive to KV cache compression. This suggests that when a task is heavily based on CoT to achieve better performance, compression of these crucial prompt elements has a more substantial impact on model performance. In contrast, tasks like *World Knowledge*, where the performance gain from CoT is smaller, show more resilience to KV cache compression, likely because the model relies more on its inherent knowledge than on the specific examples in the prompt.

Table 2: Zero-shot vs Few-shot Performance Comparison

Benchmark	Zero-shot $\uparrow$	CoT $\uparrow$	Delta Δ
<i>Arithmetic Reasoning</i>	29.04	79.45	+50.41
<i>World Knowledge</i>	62.62	68.82	+6.20

Observation 6. Long-Context Generation Sensitivity: KV cache compression exhibits significant performance degradation in long-context generation tasks. As demonstrated in Table 3, our evaluation of three unified compression methods—StreamingLLM, H2O, and PyramidInfer—on *LG-GSM8K* reveals substantial performance limitations. In this arithmetic reasoning task with approximately 4k token generation duration, compression methods show notable deterioration, with performance declining by more than 20% at compression ratios below 30%. The ShotKV is our proposed method that aims to improve the performance of KV cache compression on Long-Context Generation tasks, details in Section 4.

Table 3: KV cache compression methods’ performance on *LG-GSM8K*

Method	100%	40%	35%	30%	25%
FullKV	46.00	-	-	-	-
StreamingLLM	-	39.50	28.67	14.83	6.33
H2O	-	32.66	25.17	19.83	14.83
PyramidInfer	-	38.33	27.67	20.50	16.67
ShotKV(Ours)	-	47.33	41.33	38.33	26.83

4 ShotKV

Our comprehensive empirical investigation in Section 3.3 has systematically revealed critical vulnerabilities in current KV cache compression approaches when applied to a diverse range of fundamental LLM capabilities. Key findings indicate that:

- Specific task categories, notably *Arithmetic Reasoning* (**Observation 1**) and *Long-Context Generation* (**Observation 6**), exhibit pronounced performance degradation under aggressive compression.
- The integrity of prompt information is paramount; tasks that derive significant benefits from ICL and CoT (**Observation 5**) or rely heavily on n-shot prompts (as evidenced by the attention patterns in Figure 2) are particularly susceptible to information loss from compression.
- Preserving semantic coherence is crucial, with chunk-level strategies showing promise in complex reasoning tasks (**Observation 4**), suggesting that compressing or discarding tokens without regard to these semantic units can be detrimental.

These observations collectively underscore the limitations of existing unified compression methods, which often fail to preserve nuanced structured information embedded in prompts, thereby leading to documented performance drops. This necessitates a more discerning compression strategy that is acutely aware of the semantic and structural importance of prompt components, especially for tasks demanding intricate reasoning and extensive generation.

To address these multifaceted challenges identified by our empirical study, we introduce ShotKV, a novel decoding-time compression method. ShotKV is specifically designed to mitigate the observed performance degradation by strategically managing KV cache during the prefill and decoding phases. Our approach is founded on the principle that n-shot examples in prompts are not merely token sequences, but constitute coherent semantic chunks vital for effective reasoning (a concept supported by Figure 2 and Observation 4). We therefore design ShotKV to preserve these shot examples intact during the prefill phase, complemented by a distinct strategy for the decoding phase, aiming for robust performance, particularly on the sensitive tasks highlighted in our analysis.

4.1 Implementation

The **ShotKV** (Prefill-Decoding Separated **Shot**-aware **KV** Cache Compression), which separates the compression strategy for prefill and decoding phases. The key insight is that the prefill phase KV cache, which contains crucial prompt information, should be compressed once and remain fixed, while the decoding phase KV cache can be dynamically compressed with different strategies.

Given a prompt with n shots and tokens generated, we define:

$$KV_{\text{total}} = KV_{\text{prefill}} \cup KV_{\text{decoding}} \quad (2)$$

For the prefill phase, we compute shot importance and preserve complete shot examples:

$$\text{Score}_{\text{prefill}}(s_i) = \frac{1}{k_i} \sum_{t \in s_i} \sum_{h=1}^H \sum_{l=1}^L \alpha_{t,h}^l \quad (3)$$

where s_i represents the i -th shot example containing k_i tokens. The term $\alpha_{t,h}^l$ denotes the attention weight assigned by the query vector (corresponding to the first token to be decoded immediately following the prompt) to the key vector of a token t within shot s_i , in attention head h at transformer layer l . Once the prefill phase KV cache is compressed based on these scores, it remains fixed throughout the generation process.

Given a prefill compression ratio r_p , we prioritize shots with higher scores while ensuring that the total number of preserved tokens does not exceed the KV cache limit. Specifically, the shots are ranked by their scores and selected in descending order until they reach the compression budget $r_p \times |KV_{\text{prefill}}|$. This shot-level selection strategy helps to maintain the semantic coherence of important examples while adhering to memory constraints.

$$KV_{\text{prefill}}^C = \text{Compress}(\{s_i | s_i \in S_{\text{preserved}}^*\}) \quad (4)$$

$$\text{where } S_{\text{preserved}} = \underset{S \subseteq \{s_1, \dots, s_n\}}{\text{argmax}} \sum_{s_i \in S} \text{Score}_{\text{prefill}}(s_i) \quad (5)$$

$$\text{subject to: } \sum_{s_i \in S} k_i \leq r_p \times |KV_{\text{prefill}}| \quad (6)$$

Here, KV_{prefill}^C represents the compressed KV cache for prefilling and $S_{\text{preserved}}$ represents the optimal subset of shots that should be preserved after compression. The first equation aims to maximize the total importance score of the selected shots, where $\{s_1, \dots, s_n\}$ represents all available shots and $\text{Score}_{\text{prefill}}(s_i)$ is the importance score of the shot s_i calculated using attention weights as defined earlier. The second equation enforces the memory constraint: the total number of tokens (k_i) in the selected shots must not exceed the allocated budget, which is determined by the prefill compression ratio r_p multiplied by the original KV cache size.

For the decoding phase, we compute importance scores only for the tokens generated during decoding:

$$\text{Score}_{\text{decoding}}(t) = \sum_{h=1}^H \sum_{l=1}^L \alpha_{t,h}^l \quad (7)$$

Here, for a previously generated token t , $\alpha_{t,h}^l$ is similarly defined as the attention weight assigned by the query vector of the current token being generated to the key vector of token t , within head h at layer l . Thus, $\text{Score}_{\text{decoding}}(t)$ represents the total attention received by token t from the current generation step.

Given a decoding compression ratio r_d , we select the tokens with the highest scores to preserve. The compressed decoding KV cache KV_{decoding}^C retains only the top- k tokens where $k = r_d \times |KV_{\text{decoding}}|$, effectively maintaining the most influential context tokens while reducing memory usage:

$$KV_{\text{decoding}}^C = \text{TopK}(KV_{\text{decoding}}, \text{Score}_{\text{decoding}}, k = r_d \times |KV_{\text{decoding}}|) \quad (8)$$

Finally, we combine compressed prefill and decoding KV caches to form the total compressed KV cache:

$$KV_{\text{total}} = KV_{\text{prefill}}^C \cup KV_{\text{decoding}}^C \quad (9)$$

4.2 Empirical Results

In this section, we evaluate ShotKV under two scenarios: many-shot *Arithmetic Reasoning* with multiple KV cache compression methods, and *LG-GSM8K* with three unified compression methods that optimize the KV cache during generation.

Baseline. For *LG-GSM8K* evaluation, we employ three state-of-the-art unified compression methods as baselines: StreamingLLM [10], H2O [21], and PyramidInfer [26]. We conduct experiments using LLaMA-3-8B-Instruct [39] on the *LG-GSM8K* benchmark [38], maintaining consistent parameters with Observation 6 ($K = 35$, $T = 20$). For many-shot *Arithmetic Reasoning* experiments, we follow the configuration detailed in Observation 4.

Main results and analysis. From the Table 4, we can see that ShotKV achieves the best performance on *LG-GSM8K*, maintaining high performance at low compression ratios. Specifically, at a compression ratio of 40%, ShotKV achieves 47.33% accuracy, surpassing the full kv cache baseline (46.00%) and showing substantial improvements over other methods (32.66%-39.50%). And Table 3 shows that ShotKV also achieves the best performance on many-shot *Arithmetic Reasoning*, maintaining high performance at low compression ratios. Even in aggressive compression ratios (25%-30%), ShotKV maintains relatively stable performance (26.83%-38.33%), while other methods experience more severe degradation (6.33%-16.67%). This superior performance can be attributed to two key design choices: (1) the preservation of complete shot examples during compression maintains the semantic coherence necessary for mathematical reasoning, and (2) the separation of prefill and decoding phase compression allows for more flexible and task-appropriate token retention strategies. These results suggest that our shot-aware compression strategy is particularly effective for long-context generation tasks that require maintaining complex reasoning chains, such as mathematical problem solving.

Table 4: KV cache compression methods’ performance on Many-shot *Arithmetic Reasoning*

Method	100%	40%	30%	20%	10%
FullKV	82.35	-	-	-	-
StreamingLLM	-	80.37	78.35	75.37	74.32
H2O	-	78.32	79.32	74.28	51.27
PyramidKV	-	78.34	79.34	78.32	70.37
SnapKV	-	79.35	80.38	79.34	68.27
ChunkKV	-	78.32	79.32	78.35	79.32
ShotKV(Ours)	-	81.07	80.82	80.57	80.37

5 Conclusion

This paper presents KVFundaBench, a benchmark for systematically evaluating the effects of KV cache compression on various fundamental LLM capabilities. Our findings reveal that performance degradation is highly task dependent, with arithmetic reasoning and long-context generation being particularly sensitive (*Task-Dependent Degradation* and *Long-Context Generation Sensitivity*). We also highlight that compression sensitivity is influenced by a confluence of factors, including inherent model characteristics such as training dynamics (*Model-Type Robustness*), prompt-level attributes like length (*Prompt Length Vulnerability*), and the reliance on in-context examples (*Prompt-Gain Sensitivity*). Crucially, we demonstrate the importance of preserving the semantic integrity of prompt components, especially at a chunk or shot level, for complex reasoning and generation tasks where current methods often struggle and where chunk-based approaches show promise (*Chunk-Level Superiority*).

Based on these insights, we introduced ShotKV, a novel compression framework that distinctively manages prefill and decoding phases while prioritizing shot-level semantic coherence to mitigate information loss in sensitive tasks. ShotKV demonstrates superior performance, notably on long-context arithmetic reasoning and generation tasks, maintaining high accuracy even at aggressive compression ratios. The results of KVFundaBench and the efficacy of ShotKV underscore the potential for more nuanced compression strategies and suggest promising future research avenues.

References

- [1] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21:140:1–140:67, 2020. URL <http://jmlr.org/papers/v21/20-074.html>.
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [3] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *ArXiv preprint*, abs/2204.02311, 2022. URL <https://arxiv.org/abs/2204.02311>.
- [4] Yi Tay, Mostafa Dehghani, Vinh Q Tran, Xavier Garcia, Dara Bahri, Tal Schuster, Huaixiu Steven Zheng, Neil Houlsby, and Donald Metzler. Unifying language learning paradigms. *ArXiv preprint*, abs/2205.05131, 2022. URL <https://arxiv.org/abs/2205.05131>.
- [5] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *ArXiv preprint*, abs/2302.13971, 2023. URL <https://arxiv.org/abs/2302.13971>.
- [6] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *ArXiv preprint*, abs/2307.09288, 2023. URL <https://arxiv.org/abs/2307.09288>.
- [7] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.
- [8] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.
- [9] Sam Ade Jacobs et al. DeepSpeed Ulysses: System optimizations for enabling training of extreme long sequence Transformer models. *ArXiv preprint*, abs/2309.14509, 2023. URL <https://arxiv.org/abs/2309.14509>.
- [10] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=NG7sS51zVF>.
- [11] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *ArXiv preprint*, abs/2306.15595, 2023. URL <https://arxiv.org/abs/2306.15595>.
- [12] Wenhan Xiong, Jingyu Liu, Igor Molybog, Hejia Zhang, Prajjwal Bhargava, Rui Hou, Louis Martin, Rashi Rungta, Karthik Abinav Sankararaman, Barlas Oguz, Madian Khabsa, Han Fang, Yashar Mehdad, Sharan Narang, Kshitiz Malik, Angela Fan, Shruti Bhosale, Sergey Edunov, Mike Lewis, Sinong Wang, and Hao Ma. Effective long-context scaling of foundation models. In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 4643–4663, Mexico City, Mexico, 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.naacl-long.260>.
- [13] Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. Longlora: Efficient fine-tuning of long-context large language models. In *The Twelfth International Conference on Learning Representations*, 2023.
- [14] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=wHBfxhZu1u>.

- [15] AI21. Introducing jamba: Ai21’s groundbreaking ssm-transformer model, 2024. URL <https://www.ai21.com/blog/announcing-jamba>.
- [16] X.AI. Announcing grok-1.5, 2024. URL <https://x.ai/blog/grok-1.5>.
- [17] Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy Lillicrap, Jean-baptiste Alayrac, Radu Soricut, Angeliki Lazaridou, Orhan Firat, Julian Schrittwieser, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *ArXiv preprint*, abs/2403.05530, 2024. URL <https://arxiv.org/abs/2403.05530>.
- [18] Anthropic. Introducing the next generation of claude, 2024. URL <https://www.anthropic.com/news/claude-3-family>.
- [19] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024.
- [20] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [21] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710, 2023.
- [22] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. *ArXiv preprint*, abs/2404.14469, 2024. URL <https://arxiv.org/abs/2404.14469>.
- [23] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms. *ArXiv preprint*, abs/2310.01801, 2023. URL <https://arxiv.org/abs/2310.01801>.
- [24] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, et al. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.
- [25] Qichen Fu, Minsik Cho, Thomas Merth, Sachin Mehta, Mohammad Rastegari, and Mahyar Najibi. LazyLLM: Dynamic token pruning for efficient long context LLM inference. In *Workshop on Efficient Systems for Foundation Models II @ ICML2024*, 2024. URL <https://openreview.net/forum?id=gGZD1dsJqZ>.
- [26] Dongjie Yang, XiaoDong Han, Yan Gao, Yao Hu, Shilin Zhang, and Hai Zhao. Pyramid-infer: Pyramid kv cache compression for high-throughput llm inference. *arXiv preprint arXiv:2405.12532*, 2024.
- [27] Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant Nair, Ilya Soloveychik, and Purushotham Kamath. Keyformer: Kv cache reduction through key tokens selection for efficient generative inference. *Proceedings of Machine Learning and Systems*, 6:114–127, 2024.
- [28] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. *Advances in Neural Information Processing Systems*, 36, 2024.
- [29] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. *ArXiv preprint*, abs/2406.10774, 2024. URL <https://arxiv.org/abs/2406.10774>.
- [30] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023.
- [31] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks, 2025. URL <https://arxiv.org/abs/2412.15204>.
- [32] Gregory Kamradt. Needle In A Haystack - pressure testing LLMs. *Github*, 2023. URL https://github.com/gkamradt/LLMTest_NeedleInAHaystack/tree/main.

- [33] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- [34] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukas Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [35] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In Jill Burstein, Christy Doran, and Tamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4149–4158, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1421. URL <https://aclanthology.org/N19-1421/>.
- [36] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [37] Weidi Luo, Siyuan Ma, Xiaogeng Liu, Xiaoyu Guo, and Chaowei Xiao. Jailbreakv: A benchmark for assessing the robustness of multimodal large language models against jailbreak attacks. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=GC4mXVfquq>.
- [38] Xiang Liu, Peijie Dong, Xuming Hu, and Xiaowen Chu. LongGenBench: Long-context generation benchmark. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 865–883, Miami, Florida, USA, November 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.findings-emnlp.48>.
- [39] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [40] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- [41] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [42] Xiang Liu, Zhenheng Tang, Peijie Dong, Zeyu Li, Bo Li, Xuming Hu, and Xiaowen Chu. Chunkkv: Semantic-preserving kv cache compression for efficient long-context llm inference, 2025. URL <https://arxiv.org/abs/2502.00299>.
- [43] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 12 2023. URL <https://zenodo.org/records/10256836>.
- [44] Rishabh Agarwal, Avi Singh, Lei M Zhang, Bernd Bohnet, Luis Rosias, Stephanie Chan, Biao Zhang, Ankesh Anand, Zaheer Abbas, Azade Nova, et al. Many-shot in-context learning. *arXiv preprint arXiv:2404.11018*, 2024.
- [45] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5:606–624, 2023.
- [46] Zhiyu Guo, Hidetaka Kamigaito, and Taro Watanabe. Attention score is not all you need for token importance indicator in kv cache reduction: Value also matters. *arXiv preprint arXiv:2406.12335*, 2024.
- [47] Yuxin Zhang, Yuxuan Du, Gen Luo, Yunshan Zhong, Zhenyu Zhang, Shiwei Liu, and Rongrong Ji. Cam: Cache merging for memory-efficient llms inference. In *Forty-first International Conference on Machine Learning*, 2024.

- [48] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. Cacheblend: Fast large language model serving with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.
- [49] Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S Kevin Zhou. Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference. *arXiv preprint arXiv:2407.11550*, 2024.
- [50] Jialong Wu, Zhenglin Wang, Linhai Zhang, Yilong Lai, Yulan He, and Deyu Zhou. Scope: Optimizing key-value cache compression in long-context generation, 2024. URL <https://arxiv.org/abs/2412.13649>.
- [51] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*, 2022.
- [52] Haoyi Wu and Kewei Tu. Layer-condensed kv cache for efficient inference of large language models, 2024. URL <https://arxiv.org/abs/2405.10637>.
- [53] Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. You only cache once: Decoder-decoder architectures for language models. *arXiv preprint arXiv:2405.05254*, 2024.
- [54] William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan Kelly. Reducing transformer key-value cache size with cross-layer attention. *arXiv preprint arXiv:2405.12981*, 2024.
- [55] Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. Mini-cache: Kv cache compression in depth dimension for large language models. *arXiv preprint arXiv:2405.14366*, 2024.
- [56] David Wingate, Mohammad Shoeybi, and Taylor Sorensen. Prompt compression and contrastive conditioning for controllability and toxicity reduction in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 5621–5634, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.findings-emnlp.412. URL <https://aclanthology.org/2022.findings-emnlp.412>.
- [57] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3829–3846, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.232. URL <https://aclanthology.org/2023.emnlp-main.232>.
- [58] Wangchunshu Zhou, Yuchen Eleanor Jiang, Peng Cui, Tiannan Wang, Zhenxin Xiao, Yifan Hou, Ryan Cotterell, and Mrinmaya Sachan. Recurrentgpt: Interactive generation of (arbitrarily) long text, 2023.
- [59] Qingyue Wang, Liang Ding, Yanan Cao, Zhiliang Tian, Shi Wang, Dacheng Tao, and Li Guo. Recursively summarizing enables long-term dialogue memory in large language models. *arXiv preprint arXiv:2308.15022*, 2023.
- [60] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMlingua: Compressing prompts for accelerated inference of large language models. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.825. URL <https://aclanthology.org/2023.emnlp-main.825>.
- [61] Huiqiang Jiang, Qianhui Wu, , Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677, Bangkok, Thailand, August 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.acl-long.91>.

- [62] Weizhi Fei, Xueyan Niu, Pingyi Zhou, Lu Hou, Bo Bai, Lei Deng, and Wei Han. Extending context window of large language models via semantic compression. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics ACL 2024*, pages 5169–5181, Bangkok, Thailand and virtual meeting, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.306. URL <https://aclanthology.org/2024.findings-acl.306>.
- [63] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- [64] Aida Amini, Saadia Gabriel, Peter Lin, Rik Koncel-Kedziorski, Yejin Choi, and Hannaneh Hajishirzi. Mathqa: Towards interpretable math word problem solving with operation-based formalisms. *arXiv preprint arXiv:1905.13319*, 2019.
- [65] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615*, 2022.
- [66] Jiayi Yuan, Hongyi Liu, Shaochen Zhong, Yu-Neng Chuang, Songchen Li, Guanchu Wang, Duy Le, Hongye Jin, Vipin Chaudhary, Zhaozhuo Xu, et al. Kv cache compression, but what must we give in return? a comprehensive benchmark of long context capable approaches. *arXiv preprint arXiv:2407.01527*, 2024.
- [67] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. *arXiv preprint arXiv:2109.07958*, 2021.
- [68] Thomas Hartvigsen, Saadia Gabriel, Hamid Palangi, Maarten Sap, Dipankar Ray, and Ece Kamar. Toxigen: A large-scale machine-generated dataset for adversarial and implicit hate speech detection. *arXiv preprint arXiv:2203.09509*, 2022.
- [69] Paul Pu Liang, Chiyu Wu, Louis-Philippe Morency, and Ruslan Salakhutdinov. Towards understanding and mitigating social biases in language models. In *International Conference on Machine Learning*, pages 6565–6576. PMLR, 2021.
- [70] Kaijie Zhu, Jindong Wang, Jiaheng Zhou, Zichen Wang, Hao Chen, Yidong Wang, Linyi Yang, Wei Ye, Yue Zhang, Neil Zhenqiang Gong, et al. Promptbench: Towards evaluating the robustness of large language models on adversarial prompts. *arXiv e-prints*, pages arXiv–2306, 2023.
- [71] Xinyue Shen, Zeyuan Chen, Michael Backes, Yun Shen, and Yang Zhang. ”do anything now”: Characterizing and evaluating in-the-wild jailbreak prompts on large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 1671–1685, 2024.
- [72] Yue Deng, Wenxuan Zhang, Sinno Jialin Pan, and Lidong Bing. Multilingual jailbreak challenges in large language models. *arXiv preprint arXiv:2310.06474*, 2023.
- [73] Qi Li, Xiang Liu, Zhenheng Tang, Peijie Dong, Zeyu Li, Xinglin Pan, and Xiaowen Chu. Should we really edit language models? on the evaluation of edited language models. *arXiv preprint arXiv:2410.18785*, 2024.
- [74] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [75] Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, et al. ∞ -bench: Extending long context evaluation beyond 100k tokens. *ArXiv preprint*, abs/2402.13718, 2024. URL <https://arxiv.org/abs/2402.13718>.
- [76] Uri Shaham, Maor Ivgi, Avia Efrat, Jonathan Berant, and Omer Levy. ZeroSCROLLS: A zero-shot benchmark for long text understanding. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 7977–7989, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.536. URL <https://aclanthology.org/2023.findings-emnlp.536>.

- [77] Chenxin An, Shansan Gong, Ming Zhong, Mukai Li, Jun Zhang, Lingpeng Kong, and Xipeng Qiu. L-eval: Instituting standardized evaluation for long context language models. *ArXiv preprint*, abs/2307.11088, 2023. URL <https://arxiv.org/abs/2307.11088>.
- [78] Amirkeivan Mohtashami and Martin Jaggi. Landmark attention: Random-access infinite context length for transformers. *ArXiv preprint*, abs/2305.16300, 2023. URL <https://arxiv.org/abs/2305.16300>.
- [79] Dacheng Li, Rulin Shao, et al. How long can open-source LLMs truly promise on context length?, 2023. URL <https://lmsys.org/blog/2023-06-29-longchat>.
- [80] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl.a.00638. URL <https://aclanthology.org/2024.tacl-1.9>.
- [81] Cheng-Ping Hsieh, Simeng Sun, Samuel Krman, Shantanu Acharya, Dima Rekish, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What’s the real context size of your long-context language models? *ArXiv preprint*, abs/2404.06654, 2024. URL <https://arxiv.org/abs/2404.06654>.
- [82] Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. Long range arena : A benchmark for efficient transformers. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL <https://openreview.net/forum?id=qVyeW-grC2k>.

Appendix

A	Related Work	17
B	Experiment Details	19
B.1	Hyper-parameters	19
B.2	Detail Results	19
B.3	More experiments on other models	20
C	ShotKV	21
C.1	Pseudocode	21
C.2	Latency and Throughput	23
D	Evaluation Benchmark	25
D.1	Dataset Details	25
E	Impact Statement	25
F	Limitations	25

A Related Work

Key-value Cache Optimization Techniques KV cache is the core component in LLM inference, which avoids repetitive computations by caching Key and Value vectors. However, the cost of caching KV increases exponentially with the expansion of the model size and the length of the context [45]. Some approaches have been published to alleviate the problem. For example, KV Compression designs efficient content selection strategies to filter and manage tokens [21, 27]. Some methods identify important tokens by focusing on high attention allocation [22], while others optimize token selection by combining attention scores with value vector norms to improve importance evaluation [46]. Techniques like PyramidInfer reduce critical contexts layer by layer based on the distribution of attention scores [26], and StreamingLLM preserves attention sinks to maintain stable performance in extended sequences [10]. Researchers reduce storage costs by merging similar context representations and solving input disturbances caused by compression [47]. For example, CaM [47] works by integrating the KV cache to be dropped into the retained cache in proportion to the attention weight. In addition, Yao et al. [48] proposes CacheBlend to achieve a selective KV recompute. Only partial KVs of crucial tokens are updated to reduce the delay in the prefill stage and increase the throughput. In addition, the dynamic budget allocation method is also used to optimize the KV cache, which adjusts the resource allocation in real time according to the importance of the context, providing a balance between performance and efficiency in multitask inference scenarios [24, 49]. Wu et al. [50] proposes a prefill-decoding separation strategy to optimize the KV cache compression.

Evaluation of LLMs’ Fundamental Abilities Accurately evaluating the fundamental capabilities of large language models is crucial to understand their true potential and limitations. The evaluation typically spans across several key dimensions: world knowledge tasks like MMLU [33], BBH [51] assess models’ grasp of diverse domains through multiple-choice questions; commonsense reasoning tasks such as CSQA [35] evaluate inference and context understanding abilities; arithmetic reasoning benchmarks like GSM8K [34] test mathematical problem-solving capabilities through step-by-step reasoning; code generation tasks including HumanEval [36] measure the ability to generate functionally correct code; and safety evaluations using benchmarks like JailBreakV [37] assess models’ robustness against harmful content generation. Additionally, long-context benchmarks such as LongBench [30, 31] and Need-In-A-Haystack (NIAH) [32] aiming to evaluate models’ long-context summarization and retrieval capabilities. Furthermore, LongGenBench [38] evaluates the models’ ability to process and generate responses for extended input sequences. And recently, in-context many-shot learning has been recognized as a long-context reasoning paradigm [44], which considers the number of shots as a critical factor in the performance of LLM. Although these tasks typically employ automatic evaluation metrics for standardization, KV cache compression may introduce unique challenges, particularly in tasks requiring complex reasoning chains or extensive knowledge retrieval.

KV cache sharing Recent work has explored various strategies for sharing KV caches across transformer layers. The Layer Condensed KV Cache (LCKV) [52] computes the KV only for the top layer and pairs them with queries from all layers, while optionally retaining standard attention for a few top and bottom layers to mitigate performance degradation. Similarly, You Only Cache Once (YOCO) [53] computes KVs exclusively for the top layer but pairs them with queries from only the top half of layers, employing efficient attention in the bottom layers to maintain a constant cache size. In contrast, Cross-Layer Attention (CLA) [54] divides layers into groups, pairing queries from all layers in each group with KVs from that group’s bottom layer. MiniCache [55] introduces a novel method that merges KV caches in layering while enabling recovery during compute-in-place operations, optimizing the size of the KV cache. These methods illustrate various trade-offs between computation, memory usage, and model performance when sharing KV caches across transformer layers.

Prompting Compression Recent advances in prompt compression have yielded innovative approaches to information density optimization in natural language processing. Research by Wingate et al. [56] demonstrates how soft prompting techniques can achieve higher information density per token. Building upon this foundation, AutoCompressor [57] leverages soft prompts to both condense input sequences and expand model context windows. Parallel developments by Zhou et al. [58] and Wang et al. [59] showcase iterative summarization strategies using LLMs, establishing persistent memory mechanisms particularly beneficial for narrative construction and conversational systems.

The progressive development of the LLMingua framework [60–62] has advanced prompt compression capabilities across extended context processing, logical reasoning, and retrieval-augmented generation. Notable contributions from Fei et al. [62] demonstrate effective context management through automated segmentation and semantic condensation using pre-trained language models.

General Tasks General tasks refer to evaluating the overall performance of LLMs under mathematical inference, logic reasoning, and common knowledge. GSM8K [34] and MMLU [33] are representative tasks. The former focuses on the step-by-step reasoning ability of mathematical problem solving, while the latter covers assessment of common sense and expertise in multiple areas. Besides, MATH [63] spans various mathematical fields, ranging from elementary algebra to calculus, aiming to improve the mathematical problem-solving capabilities of LLMs. Meanwhile, MathQA [64] is a large-scale dataset comprising approximately 37,000 multiple-choice questions with precise annotations, designed to enhance the interpretability and performance of LLMs. In addition, BBH [51], a subset of BIG-Bench [65], focuses on challenging tasks. BBH includes multi-step reasoning problems, highlighting the importance of Chain-of-Thought prompting in LLMs. Similarly, CSQA [35] is a task that combines knowledge graph-based multi-step reasoning with conversational capabilities. CSQA emphasizes inference and context understanding grounded in knowledge graphs. Normally, the general tasks apply automatic evaluation metrics (e.g. multi-choice accuracy) to ensure comparability and standardization. However, optimization strategies like KV cache compression may introduce challenges in executing the mentioned tasks. Filtering and dropping of contexts are involved in the compression strategy which may lead to an intermediate inference steps missing. In addition, in tasks such as MMLU that are highly dependent on knowledge coverage, compression may weaken the model’s ability to capture long context or rare domain knowledge [66].

Security Tasks Security tasks focus on assessing the robustness and protections of LLMs against harmful content, including truthfulness [67], toxicity [68], and bias [69]. Recently, researchers noticed the weakness of LLMs in adversarial prompts [70], especially in generating illegal or inappropriate content under jailbreak prompts. Shen et al. [71] analyze the jailbreak prompts in real cases to reveal the failure of model security mechanism under complex malicious input. Meanwhile, Deng et al. [72] demonstrates the multilingual jailbreak makes model security in low-resource languages easier to bypass, significantly increasing the probability that users of low-resource languages will generate insecure content. Similar to general tasks, KV optimization techniques can cause the model to ignore potential security threats when dealing with jailbreak prompts, thereby improving the success rate of adversarial prompts [73].

Code Generation Tasks Code generation tasks test the capacities of LLMs to generate code, which not only requires that the model can generate syntactic code based on natural language description but also has certain logical reasoning abilities. HumanEval [36] and MBPP [74] are the commonly used benchmarks. They measure the functional correctness of the model by testing the results of the code’s execution.

Long-context Tasks Recent developments in evaluating long-context models have produced a comprehensive ecosystem of benchmarks, focusing on both comprehension depth and retrieval efficiency. In the comprehension domain, ∞ -Bench [75] has established new standards by crafting evaluation scenarios exceeding 100,000 tokens, while LongBench [30, 31] introduced multilingual assessment frameworks spanning document comprehension, text synthesis, and programming tasks. Further enriching this landscape, ZeroSCROLLS [76] and L-Eval [77] have expanded evaluation criteria to encompass real-world applications, particularly in query-based content summarization. The emergence of many-shot learning as a distinct paradigm for extended context processing [44] has added another dimension to this field. Notable contributions from LongGenBench [38] have advanced evaluation methodologies by combining extensive response generation requirements with efficient, cost-effective quality metrics.

The development of retrieval-focused benchmarks has taken a distinct approach, predominantly utilizing constructed datasets that enable precise experimental control, particularly in managing input sequence lengths. This methodology helps neutralize variations in model performance stemming from differences in training approaches. Substantial research efforts have yielded specialized synthetic frameworks for assessing retrieval capabilities [32, 78–81], while concurrent investiga-

tions have revealed the broader implications of extended context processing for enhanced reasoning capabilities [82].

B Experiment Details

B.1 Hyper-parameters

The hyper-parameters for different observations are shown in Table 5. The temperature for the experiments are set to 0 for ensuring the deterministic results.

Table 5: Hyperparameters for Different Observations

Benchmarks	Obs 1	Obs 2	Obs 3	Obs 4	Obs 5	Obs 6	
	Number of Shots					K	T
MMLU [33]	5	5	-	-	0,5	-	-
CommonsenseQA [35]	4	4	-	-	-	-	-
GSM8K [34]	8	8	1-8	50	0,8	-	-
HumanEval [36]	8	8	-	-	-	-	-
JailBreakV [37]	8	8	-	-	-	-	-
LongGenBench-GSM8K [38]	-	-	-	-	-	35	20

B.2 Detail Results

This section provide the detailed results of experiments in this paper, the results are shown in the format of x_y , where x is the performance of the method and y is the ΔP from the Equation (1).

Observation 1. KV cache compression methods show task-dependent performance degradation, WK and CSR are more robust to KV cache compression.

The detailed results of different KV cache compression methods are shown in Table 7, different tasks exhibit notably varied sensitivities to KV cache compression, particularly under aggressive compression ratios. At a 10% compression ratio, MMLU demonstrates remarkable resilience with less than 1% average performance degradation, while GSM8K experiences a severe average performance drop exceeding 35%. Other tasks show moderate to significant degradation, ranging from 6.5% to 17.2%. This substantial variation in compression sensitivity across tasks suggests that the effectiveness of KV cache compression is highly task-dependent, necessitating careful consideration of the specific task requirements when determining appropriate compression ratios.

The Table 6 compares the performance of R1-Distill-Llama-8B and LLaMA-3.1-8B-Instruct under different compression ratios. R1-Distill-Llama-8B demonstrates more robust performance under compression compared to LLaMA-3.1-8B-Instruct. While both models start with similar baseline performance (0.6938 vs 0.7945), R1-Distill shows significantly less performance degradation under aggressive compression. Specifically, at 30% compression ratio, R1-Distill maintains a performance of 0.6407 (-7.66%), while LLaMA-3.1-8B-Instruct drops to 0.7469 (-6.00%). The difference becomes more pronounced at 10% compression ratio, where R1-Distill achieves 0.5840 (-15.82%) compared to LLaMA-3.1-8B-Instruct’s sharp decline to 0.5143 (-35.30%). This suggests that the multi-step reasoning capabilities of R1-Distill contribute to its resilience against aggressive KV cache compression, particularly in maintaining reasoning coherence under limited context conditions.

Observation 2. Multi-step reasoning LLMs are more robust to KV cache compression. As shown in Table 8, while instruct-tuned models achieve superior baseline performance (0.7945 vs 0.5122), they demonstrate heightened sensitivity to KV cache compression. This sensitivity becomes particularly pronounced at aggressive compression ratios. At 10% compression ratio, instruct-tuned models suffer an average performance degradation of 35.3% (from 0.7945 to 0.5143), nearly double the degradation observed in non-instruct-tuned models which show a 17.2% drop (from 0.5122 to 0.4244). In contrast, R1-Distill-Llama-8B shows better resilience to compression, with only a

Table 6: Performance Comparison of Different KV Cache Compression Methods on Instruction-Tuning Model and Multi-Step Reasoning Model

Benchmark	Ratio	StreamingLLM	H2O	SnapKV	PyramidKV	ChunkKV	Average $\uparrow$
RI-AR	Baseline	R1-Distill-Llama-8B FullKV: 0.6938					
	90%	0.7167(+3.30%)	0.6900(-0.55%)	0.6933(-0.07%)	0.7100(+2.34%)	0.6867(-1.02%)	0.6993(+0.79%)
	80%	0.6867(-1.02%)	0.6933(-0.07%)	0.6933(-0.07%)	0.7067(+1.86%)	0.6767(-2.47%)	0.6913(-0.36%)
	70%	0.6933(-0.07%)	0.6633(-4.40%)	0.7100(+2.34%)	0.7100(+2.34%)	0.7000(+0.89%)	0.6953(+0.22%)
	60%	0.6833(-1.51%)	0.6900(-0.55%)	0.6900(-0.55%)	0.7133(+2.81%)	0.7067(+1.86%)	0.6967(+0.42%)
	50%	0.6700(-3.43%)	0.6967(+0.42%)	0.7067(+1.86%)	0.7000(+0.89%)	0.6867(-1.02%)	0.6920(-0.26%)
	40%	0.6767(-2.47%)	0.6800(-1.99%)	0.5967(-13.99%)	0.6967(+0.42%)	0.7133(+2.81%)	0.6727(-3.04%)
	30%	0.6600(-4.87%)	0.5900(-14.96%)	0.5833(-15.93%)	0.6700(-3.43%)	0.7000(+0.89%)	0.6407(-7.66%)
	20%	0.6200(-10.64%)	0.4933(-28.90%)	0.5633(-18.81%)	0.6833(-1.51%)	0.6533(-5.84%)	0.6026(-13.14%)
	10%	0.5167(-25.53%)	0.5567(-19.76%)	0.5767(-16.88%)	0.6267(-9.67%)	0.6433(-7.28%)	0.5840(-15.82%)
AR	Baseline	LLaMA-3.1-8B-Instruct FullKV: 0.7945					
	90%	0.7695(-3.10%)	0.7923(-0.30%)	0.7839(-1.30%)	0.7854(-1.10%)	0.7824(-1.50%)	0.7827(-1.50%)
	80%	0.7642(-3.80%)	0.7938(-0.10%)	0.7824(-1.50%)	0.7900(-0.60%)	0.7824(-1.50%)	0.7826(-1.50%)
	70%	0.7642(-3.80%)	0.7900(-0.60%)	0.7923(-0.30%)	0.7983(+0.50%)	0.7809(-1.70%)	0.7851(-1.20%)
	60%	0.7650(-3.70%)	0.7809(-1.70%)	0.7885(-0.80%)	0.7923(-0.30%)	0.7885(-0.80%)	0.7830(-1.50%)
	50%	0.7657(-3.60%)	0.7854(-1.10%)	0.7847(-1.20%)	0.7854(-1.10%)	0.7824(-1.50%)	0.7807(-1.70%)
	40%	0.7491(-5.70%)	0.7688(-3.20%)	0.7756(-2.40%)	0.7839(-1.30%)	0.7763(-2.30%)	0.7707(-3.00%)
	30%	0.7051(-11.20%)	0.7225(-9.10%)	0.7619(-4.10%)	0.7718(-2.90%)	0.7733(-2.70%)	0.7469(-6.00%)
	20%	0.6384(-19.70%)	0.6406(-19.40%)	0.6884(-13.40%)	0.7142(-10.10%)	0.7763(-2.30%)	0.6916(-13.00%)
	10%	0.4784(-39.80%)	0.4503(-43.30%)	0.5034(-36.60%)	0.4829(-39.20%)	0.6566(-17.40%)	0.5143(-35.30%)

15.82% performance drop (from 0.6938 to 0.5840) at 10% compression ratio. This pattern suggests that while instruction tuning enhances model capabilities, it also makes the model more dependent on maintaining complete context information. However, models trained with multi-step reasoning capabilities like R1-Distill demonstrate better robustness against aggressive compression, likely due to their enhanced ability to maintain reasoning coherence even with limited context.

Observation 3. Short prompt length is more sensitive to KV cache compression. As demonstrated in Table 9, the impact of KV cache compression varies significantly with the number of shots in the prompt. One-shot prompts show extreme vulnerability to aggressive compression, with performance plummeting from 0.7149 to 0.0452 (a 93.7% drop) at 10% compression ratio. This sensitivity gradually decreases as the number of shots increases. For instance, at the same compression ratio, 4-shot prompts show a 46.2% performance drop (from 0.7597 to 0.4088), while 8-shot prompts demonstrate relatively better resilience with a 35.3% reduction (from 0.7945 to 0.5143). This pattern suggests that longer prompts with more examples provide redundancy that helps maintain model performance under compression, while shorter prompts lack this buffer against information loss.

Observation 4. Chunk-level compression is more effective for long-context structured reasoning tasks. As shown in Table 10, ChunkKV demonstrates superior robustness across different compression ratios, particularly under aggressive compression settings. While other methods show significant performance degradation at 10% compression ratio (StreamingLLM: -9.8%, H2O: -37.8%, SnapKV: -17.1%, PyramidKV: -14.6%), ChunkKV maintains relatively stable performance with only a -3.7% drop. This stark contrast in performance suggests that chunk-level compression better preserves the essential contextual information needed for complex reasoning tasks. The method’s effectiveness likely stems from its ability to maintain the structural integrity of related context segments, which is particularly crucial for tasks requiring extended logical reasoning and arithmetic operations.

B.3 More experiments on other models

To further validate the generality of our findings, we also evaluate the impact of KV cache compression on a different model, Mistral-7B-Instruct. As shown in Figure 9, we observe that various KV cache compression methods lead to significant performance degradation across multiple fundamental tasks, especially under aggressive compression ratios. This result demonstrates that the reduction in foundation abilities due to KV cache compression is not limited to a specific model family, but is a general phenomenon affecting different LLM architectures.

Table 7: Performance Comparison of Different KV Cache Compression Methods on KVFundaBench

Benchmark	Ratio	StreamingLLM	H2O	SnapKV	PyramidKV	ChunkKV	Average $\uparrow$
WK	Baseline	FullKV: 0.6882					
	90%	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)
	80%	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)
	70%	0.6881(-0.01%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)
	60%	0.6881(-0.01%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)
	50%	0.6881(-0.01%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)
	40%	0.6879(-0.04%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6881(-0.01%)
	30%	0.6876(-0.09%)	0.6880(-0.03%)	0.6880(-0.03%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6880(-0.03%)
	20%	0.6859(-0.33%)	0.6878(-0.06%)	0.6880(-0.03%)	0.6882(+0.00%)	0.6882(+0.00%)	0.6876(-0.08%)
	10%	0.6787(-1.38%)	0.6852(-0.44%)	0.6831(-0.74%)	0.6882(0.00%)	0.6842(-0.58%)	0.6839(-0.63%)
AR	Baseline	FullKV: 0.7945					
	90%	0.7695(-3.10%)	0.7923(-0.30%)	0.7839(-1.30%)	0.7854(-1.10%)	0.7824(-1.50%)	0.7827(-1.50%)
	80%	0.7642(-3.80%)	0.7938(-0.10%)	0.7824(-1.50%)	0.7900(-0.60%)	0.7824(-1.50%)	0.7826(-1.50%)
	70%	0.7642(-3.80%)	0.7900(-0.60%)	0.7923(-0.30%)	0.7983(+0.50%)	0.7809(-1.70%)	0.7851(-1.20%)
	60%	0.7650(-3.70%)	0.7809(-1.70%)	0.7885(-0.80%)	0.7923(-0.30%)	0.7885(-0.80%)	0.7830(-1.50%)
	50%	0.7657(-3.60%)	0.7854(-1.10%)	0.7847(-1.20%)	0.7854(-1.10%)	0.7824(-1.50%)	0.7807(-1.70%)
	40%	0.7491(-5.70%)	0.7688(-3.20%)	0.7756(-2.40%)	0.7839(-1.30%)	0.7763(-2.30%)	0.7707(-3.00%)
	30%	0.7051(-11.20%)	0.7225(-9.10%)	0.7619(-4.10%)	0.7718(-2.90%)	0.7733(-2.70%)	0.7469(-6.00%)
	20%	0.6384(-19.70%)	0.6406(-19.40%)	0.6884(-13.40%)	0.7142(-10.10%)	0.7763(-2.30%)	0.6916(-13.00%)
	10%	0.4784(-39.80%)	0.4503(-43.30%)	0.5034(-36.60%)	0.4829(-39.20%)	0.6566(-17.40%)	0.5143(-35.30%)
CSR	Baseline	FullKV: 0.7748					
	90%	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)
	80%	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)
	70%	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)
	60%	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)
	50%	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)
	40%	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)
	30%	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)	0.7748(+0.00%)
	20%	0.7174(-7.40%)	0.7748(+0.00%)	0.7740(-0.10%)	0.7748(+0.00%)	0.7699(-0.60%)	0.7622(-1.60%)
	10%	0.6806(-12.20%)	0.7510(-3.10%)	0.7191(-7.20%)	0.7723(-0.30%)	0.7002(-9.60%)	0.7246(-6.50%)
SA	Baseline	FullKV: 0.8895					
	90%	0.8893(-0.00%)	0.8890(-0.10%)	0.8894(-0.00%)	0.8893(-0.00%)	0.8896(+0.00%)	0.8893(-0.00%)
	80%	0.8878(-0.20%)	0.8885(-0.10%)	0.8895(+0.00%)	0.8891(-0.00%)	0.8894(-0.00%)	0.8889(-0.10%)
	70%	0.8872(-0.30%)	0.8879(-0.20%)	0.8896(+0.00%)	0.8889(-0.10%)	0.8895(+0.00%)	0.8886(-0.10%)
	60%	0.8845(-0.60%)	0.8848(-0.50%)	0.8892(-0.00%)	0.8887(-0.10%)	0.8899(+0.00%)	0.8874(-0.20%)
	50%	0.8849(-0.50%)	0.8749(-1.60%)	0.8886(-0.10%)	0.8884(-0.10%)	0.8894(-0.00%)	0.8852(-0.50%)
	40%	0.8734(-1.80%)	0.8557(-3.80%)	0.8880(-0.20%)	0.8877(-0.20%)	0.8900(+0.10%)	0.8790(-1.20%)
	30%	0.8329(-6.40%)	0.8015(-9.90%)	0.8858(-0.40%)	0.8899(+0.00%)	0.8846(-0.60%)	0.8589(-3.50%)
	20%	0.6501(-26.90%)	0.7178(-19.30%)	0.8806(-1.00%)	0.8751(-1.60%)	0.8902(+0.10%)	0.8028(-9.70%)
	10%	0.5314(-40.30%)	0.6544(-26.40%)	0.8434(-5.20%)	0.8556(-3.80%)	0.8799(-1.10%)	0.7529(-15.40%)
CG	Baseline	FullKV: 0.5122					
	90%	0.5061(-1.20%)	0.5183(+1.20%)	0.5122(+0.00%)	0.5122(+0.00%)	0.5122(+0.00%)	0.5122(+0.00%)
	80%	0.5061(-1.20%)	0.5183(+1.20%)	0.5183(+1.20%)	0.5305(+3.60%)	0.5061(-1.20%)	0.5159(+0.70%)
	70%	0.5000(-2.40%)	0.5244(+2.40%)	0.5122(+0.00%)	0.5183(+1.20%)	0.5122(+0.00%)	0.5134(+0.20%)
	60%	0.5061(-1.20%)	0.5366(+4.80%)	0.5366(+4.80%)	0.5305(+3.60%)	0.5244(+2.40%)	0.5268(+2.90%)
	50%	0.4939(-3.60%)	0.5427(+6.00%)	0.5061(-1.20%)	0.4939(-3.60%)	0.4878(-4.80%)	0.5049(-1.40%)
	40%	0.4817(-6.00%)	0.5427(+6.00%)	0.5244(+2.40%)	0.4939(-3.60%)	0.5000(-2.40%)	0.5085(-0.70%)
	30%	0.4817(-6.00%)	0.5305(+3.60%)	0.5000(-2.40%)	0.4939(-3.60%)	0.4817(-6.00%)	0.4976(-2.90%)
	20%	0.4634(-9.50%)	0.5061(-1.20%)	0.4939(-3.60%)	0.4695(-8.30%)	0.4878(-4.80%)	0.4841(-5.50%)
	10%	0.3659(-28.60%)	0.4634(-9.50%)	0.4268(-16.70%)	0.4207(-17.90%)	0.4451(-13.10%)	0.4244(-17.20%)

C ShotKV

This section provides the detailed description of ShotKV.

C.1 Pseudocode

The detailed algorithm of ShotKV is presented in Algorithm1. Our method consists of two main phases: prefill compression and decoding compression. During the prefill phase, we compute an importance score for each shot by averaging the attention weights across all tokens, heads, and layers within that shot. This score $\text{Score}_{\text{prefill}}(s_i)$ is normalized by the shot length k_i to avoid bias towards longer shots. Shots are then sorted by their scores and preserved until reaching the specified prefill ratio r_p .

In the decoding phase, compression is performed dynamically at each step. For each token in the decoding KV cache, we calculate its importance score $\text{Score}_{\text{decoding}}(t)$ by summing attention weights across all heads and layers. The top-k tokens are retained based on the decoding ratio r_d . Finally, the compressed KV cache is formed by combining both the preserved prefill and decoding caches.

Table 8: KV Cache Compression Performance Comparison on *Arithmetic Reasoning* with Different Instruction Tuning Settings

Setting	Ratio	StreamingLLM	H2O	SnapKV	PyramidKV	ChunkKV	Average $\uparrow$
w/ Instruct Tuning	Baseline	FullKV: 0.7945					
	90%	0.7695 _(-3.10%)	0.7923 _(-0.30%)	0.7839 _(-1.30%)	0.7854 _(-1.10%)	0.7824 _(-1.50%)	0.7827 _(-1.50%)
	80%	0.7642 _(-3.80%)	0.7938 _(-0.10%)	0.7824 _(-1.50%)	0.7900 _(-0.60%)	0.7824 _(-1.50%)	0.7826 _(-1.50%)
	70%	0.7642 _(-3.80%)	0.7900 _(-0.60%)	0.7923 _(-0.30%)	0.7983 _(+0.50%)	0.7809 _(-1.70%)	0.7851 _(-1.20%)
	60%	0.7650 _(-3.70%)	0.7809 _(-1.70%)	0.7885 _(-0.80%)	0.7923 _(-0.30%)	0.7885 _(-0.80%)	0.7830 _(-1.50%)
	50%	0.7657 _(-3.60%)	0.7854 _(-1.10%)	0.7847 _(-1.20%)	0.7854 _(-1.10%)	0.7824 _(-1.50%)	0.7807 _(-1.70%)
	40%	0.7491 _(-5.70%)	0.7688 _(-3.20%)	0.7756 _(-2.40%)	0.7839 _(-1.30%)	0.7763 _(-2.30%)	0.7707 _(-3.00%)
	30%	0.7051 _(-11.20%)	0.7225 _(-9.10%)	0.7619 _(-4.10%)	0.7718 _(-2.90%)	0.7733 _(-2.70%)	0.7469 _(-6.00%)
	20%	0.6384 _(-19.70%)	0.6406 _(-19.40%)	0.6884 _(-13.40%)	0.7142 _(-10.10%)	0.7763 _(-2.30%)	0.6916 _(-13.00%)
	10%	0.4784 _(-39.80%)	0.4503 _(-43.30%)	0.5034 _(-36.60%)	0.4829 _(-39.20%)	0.6566 _(-17.40%)	0.5143 _(-35.30%)
w/ R1 Distill	Baseline	R1-Distill-Llama-8B FullKV: 0.6938					
	90%	0.7167 _(+3.30%)	0.6900 _(-0.55%)	0.6933 _(-0.07%)	0.7100 _(+2.34%)	0.6867 _(-1.02%)	0.6993 _(+0.79%)
	80%	0.6867 _(-1.02%)	0.6933 _(-0.07%)	0.6933 _(-0.07%)	0.7067 _(+1.86%)	0.6767 _(-2.47%)	0.6913 _(-0.36%)
	70%	0.6933 _(-0.07%)	0.6633 _(-4.40%)	0.7100 _(+2.34%)	0.7100 _(+2.34%)	0.7000 _(+0.89%)	0.6953 _(+0.22%)
	60%	0.6833 _(-1.51%)	0.6900 _(-0.55%)	0.6900 _(-0.55%)	0.7133 _(+2.81%)	0.7067 _(+1.86%)	0.6967 _(+0.42%)
	50%	0.6700 _(-3.43%)	0.6967 _(+0.42%)	0.7067 _(+1.86%)	0.7000 _(+0.89%)	0.6867 _(-1.02%)	0.6920 _(-0.26%)
	40%	0.6767 _(-2.47%)	0.6800 _(-1.99%)	0.5967 _(-13.99%)	0.6967 _(+0.42%)	0.7133 _(+2.81%)	0.6727 _(-3.04%)
	30%	0.6600 _(-4.87%)	0.5900 _(-14.96%)	0.5833 _(-15.93%)	0.6700 _(-3.43%)	0.7000 _(+0.89%)	0.6407 _(-7.66%)
	20%	0.6200 _(-10.64%)	0.4933 _(-28.90%)	0.5633 _(-18.81%)	0.6833 _(-1.51%)	0.6533 _(-5.84%)	0.6026 _(-13.14%)
	10%	0.5167 _(-25.53%)	0.5567 _(-19.76%)	0.5767 _(-16.88%)	0.6267 _(-9.67%)	0.6433 _(-7.28%)	0.5840 _(-15.82%)
w/o Instruct Tuning	Baseline	FullKV: 0.5122					
	90%	0.5061 _(-1.20%)	0.5183 _(+1.20%)	0.5122 _(+0.00%)	0.5122 _(+0.00%)	0.5122 _(+0.00%)	0.5122 _(+0.00%)
	80%	0.5061 _(-1.20%)	0.5183 _(+1.20%)	0.5183 _(+1.20%)	0.5305 _(+3.60%)	0.5061 _(-1.20%)	0.5159 _(+0.70%)
	70%	0.5000 _(-2.40%)	0.5244 _(+2.40%)	0.5122 _(+0.00%)	0.5183 _(+1.20%)	0.5122 _(+0.00%)	0.5134 _(+0.20%)
	60%	0.5061 _(-1.20%)	0.5366 _(+4.80%)	0.5366 _(+4.80%)	0.5305 _(+3.60%)	0.5244 _(+2.40%)	0.5268 _(+2.90%)
	50%	0.4939 _(-3.60%)	0.5427 _(+6.00%)	0.5061 _(-1.20%)	0.4939 _(-3.60%)	0.4878 _(-4.80%)	0.5049 _(-1.40%)
	40%	0.4817 _(-6.00%)	0.5427 _(+6.00%)	0.5244 _(+2.40%)	0.4939 _(-3.60%)	0.5000 _(-2.40%)	0.5085 _(-0.70%)
	30%	0.4817 _(-6.00%)	0.5305 _(+3.60%)	0.5000 _(-2.40%)	0.4939 _(-3.60%)	0.4817 _(-6.00%)	0.4976 _(-2.90%)
	20%	0.4634 _(-9.50%)	0.5061 _(-1.20%)	0.4939 _(-3.60%)	0.4695 _(-8.30%)	0.4878 _(-4.80%)	0.4841 _(-5.50%)
	10%	0.3659 _(-28.60%)	0.4634 _(-9.50%)	0.4268 _(-16.70%)	0.4207 _(-17.90%)	0.4451 _(-13.10%)	0.4244 _(-17.20%)

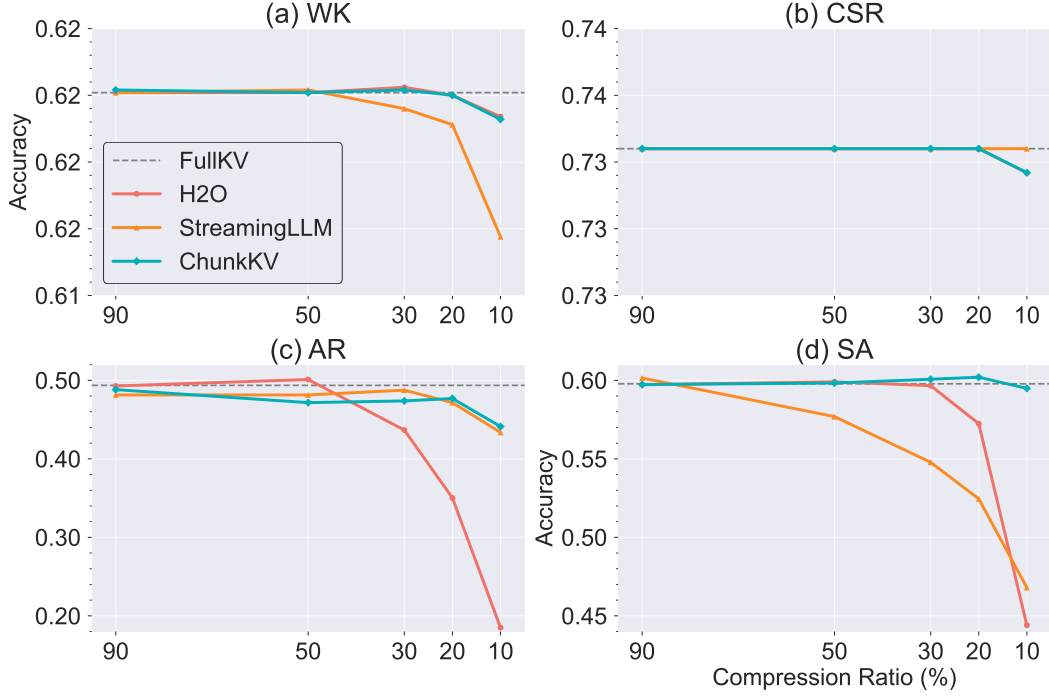


Figure 9: Performance Comparison of KV Cache Compression Methods Across Tasks with Mistral-7B-Instruct.

This two-phase approach allows for different compression strategies during prefill and decoding stages, recognizing their distinct roles in the inference process. The shot-aware design during prefill ensures that the most informative examples are preserved, while the token-level compression during decoding maintains essential recent context.

Table 9: Performance Comparison of Different Shot Numbers on GSM8K

Shot	Ratio	StreamingLLM	H2O	SnapKV	PyramidKV	ChunkKV	Average ↑
1-shot	Baseline			FullKV: 0.7149			
	90%	0.7013 _(-1.90%)	0.7172 _(+0.30%)	0.7142 _(-0.10%)	0.7020 _(-1.80%)	0.7172 _(+0.30%)	0.7104 _(-0.60%)
	80%	0.6892 _(-3.60%)	0.7089 _(-0.80%)	0.7066 _(-1.20%)	0.6952 _(-2.80%)	0.7081 _(-1.00%)	0.7016 _(-1.90%)
	70%	0.6816 _(-4.70%)	0.6914 _(-3.30%)	0.6945 _(-2.90%)	0.6884 _(-3.70%)	0.7127 _(-0.30%)	0.6937 _(-3.00%)
	60%	0.6884 _(-3.70%)	0.6831 _(-4.40%)	0.6914 _(-3.30%)	0.6816 _(-4.70%)	0.6990 _(-2.20%)	0.6887 _(-3.70%)
	50%	0.6952 _(-2.80%)	0.6596 _(-7.70%)	0.6611 _(-7.50%)	0.6717 _(-6.00%)	0.6732 _(-5.80%)	0.6722 _(-6.00%)
	40%	0.6657 _(-6.90%)	0.6202 _(-13.20%)	0.6065 _(-15.20%)	0.6475 _(-9.40%)	0.6050 _(-15.40%)	0.6290 _(-12.00%)
	30%	0.5118 _(-28.40%)	0.5004 _(-30.00%)	0.5042 _(-29.50%)	0.5898 _(-17.50%)	0.4011 _(-43.90%)	0.5015 _(-29.90%)
	20%	0.2320 _(-67.50%)	0.2714 _(-62.00%)	0.2654 _(-62.90%)	0.3973 _(-44.40%)	0.1319 _(-81.60%)	0.2596 _(-63.70%)
	10%	0.0296 _(-95.90%)	0.0243 _(-96.60%)	0.0296 _(-95.90%)	0.1236 _(-82.70%)	0.0190 _(-97.30%)	0.0452 _(-93.70%)
2-shot	Baseline			FullKV: 0.7574			
	90%	0.7544 _(-0.40%)	0.7604 _(+0.40%)	0.7574 _(+0.00%)	0.7612 _(+0.50%)	0.7627 _(+0.70%)	0.7592 _(+0.20%)
	80%	0.7551 _(-0.30%)	0.7521 _(-0.70%)	0.7559 _(-0.20%)	0.7559 _(-0.20%)	0.7589 _(+0.20%)	0.7556 _(-0.20%)
	70%	0.7521 _(-0.70%)	0.7453 _(-1.60%)	0.7566 _(-0.10%)	0.7574 _(+0.00%)	0.7642 _(+0.90%)	0.7551 _(-0.30%)
	60%	0.7475 _(-1.30%)	0.7506 _(-0.90%)	0.7521 _(-0.70%)	0.7589 _(+0.20%)	0.7695 _(+1.60%)	0.7557 _(-0.20%)
	50%	0.7460 _(-1.50%)	0.7437 _(-1.80%)	0.7437 _(-1.80%)	0.7604 _(+0.40%)	0.7619 _(+0.60%)	0.7511 _(-0.80%)
	40%	0.7445 _(-1.70%)	0.7081 _(-6.50%)	0.7202 _(-4.90%)	0.7309 _(-3.50%)	0.7650 _(+1.00%)	0.7337 _(-3.10%)
	30%	0.7506 _(-0.90%)	0.6133 _(-19.00%)	0.6657 _(-12.10%)	0.7036 _(-7.10%)	0.7445 _(-1.70%)	0.6955 _(-8.20%)
	20%	0.6217 _(-17.90%)	0.4412 _(-41.70%)	0.4936 _(-34.80%)	0.5534 _(-26.90%)	0.5368 _(-29.10%)	0.5293 _(-30.10%)
	10%	0.1516 _(-80.00%)	0.1759 _(-76.80%)	0.1622 _(-78.60%)	0.2244 _(-70.40%)	0.0735 _(-90.30%)	0.1575 _(-79.20%)
4-shot	Baseline			FullKV: 0.7597			
	90%	0.7597 _(+0.00%)	0.7604 _(+0.10%)	0.7650 _(+0.70%)	0.7642 _(+0.60%)	0.7657 _(+0.80%)	0.7630 _(+0.40%)
	80%	0.7559 _(-0.50%)	0.7688 _(+1.20%)	0.7695 _(+1.30%)	0.7680 _(+1.10%)	0.7642 _(+0.60%)	0.7653 _(+0.70%)
	70%	0.7597 _(+0.00%)	0.7695 _(+1.30%)	0.7695 _(+1.30%)	0.7710 _(+1.50%)	0.7726 _(+1.70%)	0.7682 _(+1.10%)
	60%	0.7369 _(-3.00%)	0.7726 _(+1.70%)	0.7688 _(+1.20%)	0.7635 _(+0.50%)	0.7718 _(+1.60%)	0.7627 _(+0.40%)
	50%	0.7475 _(-1.60%)	0.7612 _(+0.20%)	0.7619 _(+0.30%)	0.7665 _(+0.90%)	0.7635 _(+0.50%)	0.7601 _(+0.10%)
	40%	0.7165 _(-5.70%)	0.7339 _(-3.40%)	0.7377 _(-2.90%)	0.7483 _(-1.50%)	0.7612 _(+0.20%)	0.7395 _(-2.70%)
	30%	0.6558 _(-13.70%)	0.6603 _(-13.10%)	0.7111 _(-6.40%)	0.7263 _(-4.40%)	0.7597 _(+0.00%)	0.7026 _(-7.50%)
	20%	0.6224 _(-18.10%)	0.5625 _(-26.00%)	0.6065 _(-20.20%)	0.6543 _(-13.90%)	0.7468 _(-1.70%)	0.6385 _(-16.00%)
	10%	0.4708 _(-38.00%)	0.3980 _(-47.60%)	0.3995 _(-47.40%)	0.4321 _(-43.10%)	0.3434 _(-54.80%)	0.4088 _(-46.20%)
6-shot	Baseline			FullKV: 0.7680			
	90%	0.7551 _(-1.70%)	0.7748 _(+0.90%)	0.7839 _(+2.10%)	0.7794 _(+1.50%)	0.7794 _(+1.50%)	0.7745 _(+0.90%)
	80%	0.7642 _(-0.50%)	0.7756 _(+1.00%)	0.7809 _(+1.70%)	0.7741 _(+0.80%)	0.7786 _(+1.40%)	0.7747 _(+0.90%)
	70%	0.7513 _(-2.20%)	0.7771 _(+1.20%)	0.7809 _(+1.70%)	0.7771 _(+1.20%)	0.7786 _(+1.40%)	0.7730 _(+0.70%)
	60%	0.7468 _(-2.80%)	0.7748 _(+0.90%)	0.7733 _(+0.70%)	0.7771 _(+1.20%)	0.7809 _(+1.70%)	0.7706 _(+0.30%)
	50%	0.7407 _(-3.60%)	0.7718 _(+0.50%)	0.7718 _(+0.50%)	0.7771 _(+1.20%)	0.7718 _(+0.50%)	0.7666 _(-0.20%)
	40%	0.7377 _(-3.90%)	0.7506 _(-2.30%)	0.7771 _(+1.20%)	0.7688 _(+0.10%)	0.7854 _(+2.30%)	0.7639 _(-0.50%)
	30%	0.7058 _(-8.10%)	0.7255 _(-5.50%)	0.7392 _(-3.70%)	0.7491 _(-2.50%)	0.7763 _(+1.10%)	0.7392 _(-3.70%)
	20%	0.5921 _(-22.90%)	0.6232 _(-18.80%)	0.6732 _(-12.30%)	0.6960 _(-9.40%)	0.7665 _(-0.20%)	0.6702 _(-12.70%)
	10%	0.4572 _(-40.50%)	0.4481 _(-41.60%)	0.4958 _(-35.40%)	0.4458 _(-41.90%)	0.5565 _(-27.50%)	0.4807 _(-37.40%)
8-shot	Baseline			FullKV: 0.7945			
	90%	0.7695 _(-3.10%)	0.7923 _(-0.30%)	0.7839 _(-1.30%)	0.7854 _(-1.10%)	0.7824 _(-1.50%)	0.7827 _(-1.50%)
	80%	0.7642 _(-3.80%)	0.7938 _(-0.10%)	0.7824 _(-1.50%)	0.7900 _(-0.60%)	0.7824 _(-1.50%)	0.7826 _(-1.50%)
	70%	0.7642 _(-3.80%)	0.7900 _(-0.60%)	0.7923 _(-0.30%)	0.7983 _(+0.50%)	0.7809 _(-1.70%)	0.7851 _(-1.20%)
	60%	0.7650 _(-3.70%)	0.7809 _(-1.70%)	0.7885 _(-0.80%)	0.7923 _(-0.30%)	0.7885 _(-0.80%)	0.7830 _(-1.50%)
	50%	0.7657 _(-3.60%)	0.7854 _(-1.10%)	0.7847 _(-1.20%)	0.7854 _(-1.10%)	0.7824 _(-1.50%)	0.7807 _(-1.70%)
	40%	0.7491 _(-5.70%)	0.7688 _(-3.20%)	0.7756 _(-2.40%)	0.7839 _(-1.30%)	0.7763 _(-2.30%)	0.7707 _(-3.00%)
	30%	0.7051 _(-11.20%)	0.7225 _(-9.10%)	0.7619 _(-4.10%)	0.7718 _(-2.90%)	0.7733 _(-2.70%)	0.7469 _(-6.00%)
	20%	0.6384 _(-19.70%)	0.6406 _(-19.40%)	0.6884 _(-13.40%)	0.7142 _(-10.10%)	0.7763 _(-2.30%)	0.6916 _(-13.00%)
	10%	0.4784 _(-39.80%)	0.4503 _(-43.30%)	0.5034 _(-36.60%)	0.4829 _(-39.20%)	0.6566 _(-17.40%)	0.5143 _(-35.30%)

C.2 Latency and Throughput

We further compare the inference efficiency of ShotKV and the FullKV baseline in terms of latency and throughput under different input and output sequence lengths. As shown in Table 11, ShotKV consistently reduces latency and improves throughput compared to FullKV. For example, with an input length of 8192 and output length of 4096, ShotKV achieves an 11.3% reduction in latency and a 13.1% increase in throughput. These results demonstrate that ShotKV not only maintains model performance under aggressive KV cache compression, but also brings tangible efficiency benefits for long-context inference.

Table 10: Performance Comparison of Different KV Cache Compression Methods on Many-shot GSM8K

Benchmark	Ratio	StreamingLLM	H2O	SnapKV	PyramidKV	ChunkKV	Average $\uparrow$
Many-shot GSM8K	Baseline	LLaMA-3.1-8B-Instruct FullKV: 0.8235					
	90%	0.7728 _(-6.16%)	0.8142 _(-1.13%)	0.8137 _(-1.19%)	0.7932 _(-3.68%)	0.8233 _(-0.02%)	0.8034 _(-2.44%)
	80%	0.7935 _(-3.64%)	0.8334 _(+1.20%)	0.8138 _(-1.18%)	0.8037 _(-2.40%)	0.7932 _(-3.68%)	0.8075 _(-1.94%)
	70%	0.8038 _(-2.39%)	0.8136 _(-1.20%)	0.7832 _(-4.89%)	0.7932 _(-3.68%)	0.8037 _(-2.40%)	0.7995 _(-2.91%)
	60%	0.7932 _(-3.68%)	0.8142 _(-1.13%)	0.8037 _(-2.40%)	0.7935 _(-3.64%)	0.8038 _(-2.39%)	0.8017 _(-2.65%)
	50%	0.7934 _(-3.65%)	0.8137 _(-1.19%)	0.7932 _(-3.68%)	0.7932 _(-3.68%)	0.7835 _(-4.86%)	0.7954 _(-3.41%)
	40%	0.8037 _(-2.40%)	0.7832 _(-4.89%)	0.7935 _(-3.64%)	0.7834 _(-4.87%)	0.7832 _(-4.89%)	0.7894 _(-4.14%)
	30%	0.7835 _(-4.86%)	0.7932 _(-3.68%)	0.8038 _(-2.39%)	0.7934 _(-3.65%)	0.7932 _(-3.68%)	0.7934 _(-3.65%)
	20%	0.7537 _(-8.47%)	0.7428 _(-9.80%)	0.7934 _(-3.65%)	0.7832 _(-4.89%)	0.7835 _(-4.86%)	0.7713 _(-6.34%)
	10%	0.7432 _(-9.75%)	0.5127 _(-37.74%)	0.6827 _(-17.10%)	0.7037 _(-14.55%)	0.7932 _(-3.68%)	0.6871 _(-16.56%)
	Baseline	R1-Distill-Llama-8B FullKV: 0.7123					
	90%	0.7123 _(+1.42%)	0.6612 _(-5.85%)	0.6534 _(-6.96%)	0.6912 _(-1.58%)	0.6923 _(-1.42%)	0.6821 _(-2.88%)
	80%	0.7234 _(+3.00%)	0.6534 _(-6.96%)	0.7123 _(+1.42%)	0.6423 _(-8.54%)	0.7123 _(+1.42%)	0.6887 _(-1.94%)
	70%	0.7412 _(+5.54%)	0.6523 _(-7.12%)	0.7234 _(+3.00%)	0.6923 _(-1.42%)	0.7234 _(+3.00%)	0.7065 _(+0.60%)
	60%	0.7423 _(+5.69%)	0.6912 _(-1.58%)	0.6912 _(-1.58%)	0.6823 _(-2.85%)	0.6634 _(-5.54%)	0.6941 _(-1.17%)
	50%	0.7234 _(+3.00%)	0.7134 _(+1.58%)	0.7312 _(+4.12%)	0.7123 _(+1.42%)	0.7123 _(+1.42%)	0.7185 _(+2.31%)
	40%	0.7123 _(+1.42%)	0.6923 _(-1.42%)	0.6923 _(-1.42%)	0.7023 _(+0.00%)	0.7234 _(+3.00%)	0.7045 _(+0.31%)
	30%	0.6523 _(-7.12%)	0.7312 _(+4.12%)	0.6634 _(-5.54%)	0.7423 _(+5.69%)	0.6912 _(-1.58%)	0.6961 _(-0.88%)
	20%	0.6912 _(-1.58%)	0.5834 _(-16.93%)	0.5123 _(-27.05%)	0.6823 _(-2.85%)	0.6634 _(-5.54%)	0.6265 _(-10.79%)
	10%	0.6323 _(-9.97%)	0.5423 _(-22.78%)	0.5412 _(-22.94%)	0.5923 _(-15.66%)	0.6823 _(-2.85%)	0.5981 _(-14.84%)

Algorithm 1 ShotKV: Shot-aware KV Cache Compression

Require: Prompt with n shots $\{s_1, \dots, s_n\}$, prefill ratio r_p , decoding ratio r_d

Ensure: Compressed KV cache KV_{total}

```

1: // Phase 1: Prefill Compression (performed once)
2: for each shot  $s_i$  in  $\{s_1, \dots, s_n\}$  do
3:   Compute  $\text{Score}_{\text{prefill}}(s_i) = \frac{1}{k_i} \sum_{t \in s_i} \sum_{h=1}^H \sum_{l=1}^L \alpha_{t,h}^l$ 
4: end for
5: Sort shots by  $\text{Score}_{\text{prefill}}(s_i)$  in descending order
6:  $S_{\text{preserved}} \leftarrow$  Select shots until  $\sum_{s_i} k_i \leq r_p \times |KV_{\text{prefill}}|$ 
7:  $KV_{\text{prefill}}^C \leftarrow \text{Compress}(\{s_i | s_i \in S_{\text{preserved}}\})$ 
8: // Phase 2: Decoding Compression (performed dynamically)
9: for each decoding step do
10:  for each token  $t$  in  $KV_{\text{decoding}}$  do
11:    Compute  $\text{Score}_{\text{decoding}}(t) = \sum_{h=1}^H \sum_{l=1}^L \alpha_{t,h}^l$ 
12:  end for
13:   $k \leftarrow r_d \times |KV_{\text{decoding}}|$ 
14:   $KV_{\text{decoding}}^C \leftarrow \text{TopK}(KV_{\text{decoding}}, \text{Score}_{\text{decoding}}, k)$ 
15: end for
return  $KV_{\text{prefill}}^C \cup KV_{\text{decoding}}^C$ 

```

Table 11: Latency and throughput comparison between ShotKV and FullKV under different input-output configurations. Percentages in parentheses indicate improvements over FullKV baseline. The experiments test on the A40 server with batch size 1.

Method	Sequence Length		Performance Metrics	
	Input	Output	Latency(s) $\downarrow$	Throughput(T/S) $\uparrow$
FullKV	4096	4096	175.50	37.73
ShotKV	4096	4096	162.85 (7.2%)	41.12 (9.0%)
FullKV	8192	4096	183.42	55.93
ShotKV	8192	4096	162.78 (11.3%)	63.24 (13.1%)

D Evaluation Benchmark

D.1 Dataset Details

Detailed statistics for each benchmark dataset are provided in Table 12.

DATASET	TASK TYPE	# TEST	METRIC	EVALUATION METHOD
MMLU [33]	World Knowledge	14,079	Accuracy	Generation-Based
GSM8K [34]	Arithmetic	1,319	Exact match	Generation-Based
CSQA [35]	Commonsense	1,221	Accuracy	Generation-Based
HumanEval [36]	Code Generation	164	Pass@1 rate	Generation-Based
JailBreakV [37]	Safety	28,000	Attack success rate	Generation-Based
LongGenBench [38]	Long-Context Generation	23,000	Accuracy	Generation-Based

Table 12: The statistics of the datasets used in this paper. # TEST denote the number of training data and test data, respectively.

E Impact Statement

This work advances the field of efficient large language model deployment through systematic analysis and improvement of KV cache compression techniques. Our research has several potential societal impacts:

First, by enabling more efficient memory usage in LLMs while maintaining performance, our work contributes to reducing the computational resources and energy consumption required for AI deployment. This has positive environmental implications and makes AI technology more accessible to researchers and organizations with limited computing resources.

Second, our proposed ShotKV method specifically improves performance on long-context arithmetic reasoning tasks, which could enhance the practical applications of LLMs in education, scientific computing, and other fields requiring complex mathematical reasoning. This could lead to more reliable AI-assisted learning and problem-solving tools.

However, we acknowledge that making LLMs more efficient could accelerate their widespread adoption, potentially raising concerns about AI’s impact on employment and privacy. While our work focuses on technical improvements, we encourage the research community to carefully consider these broader implications when deploying such technologies.

We believe the benefits of more efficient and capable AI systems outweigh potential risks, particularly as our work promotes more sustainable and accessible AI development. Nevertheless, we emphasize the importance of responsible deployment and continued ethical consideration in the application of these technologies.

F Limitations

Our study has several limitations: (1) The evaluation was conducted on specific benchmarks, and performance on other tasks may vary; (2) The computational resources required for comprehensive evaluation limited our ability to test on larger models and more diverse compression methods.